Extended

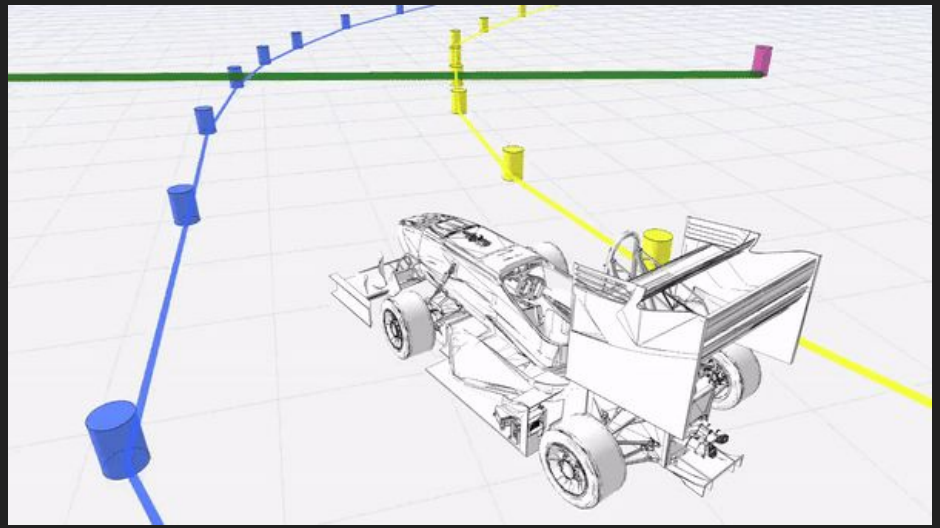
PAC SIM

Why do I even need a simulator framework?

PACSIM

Planning and controls Simulator

Developed by Elbflorance,
TU Dresden



Pros and cons

Pro:

- Open Source
- Fast and lightweight (compared to state of the art simulator)
- Integrated with ROS2
- Tailored for Formula Student

Cons:

- Non existing documentation, in ~~vino~~ code veritas
- Missing relevant parts for simulating the whole pipeline

What about perception?

Pacsim perception module:

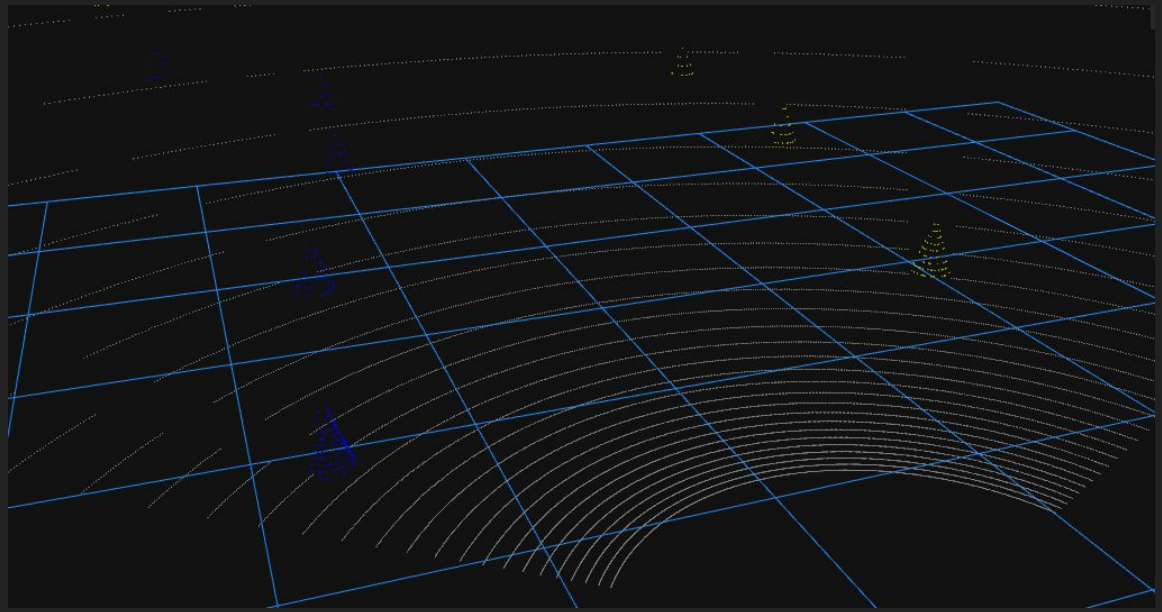
- selects cones that relies in a field of view
- add noise to the positions
- tries to classify cones (blue,yellow or orange) with probability that depend on the distance
- return the list of landmarks

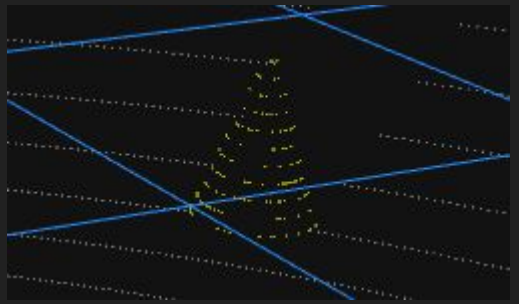
It do NOT generate any kind of point cloud like a lidar would.

You can’t simulate the perception part of the pipeline. (ground removal, clustering for cone identification…)

Lidar Simulation

Create Occlusion Array → Generate Floor Points → Generate points for each cone

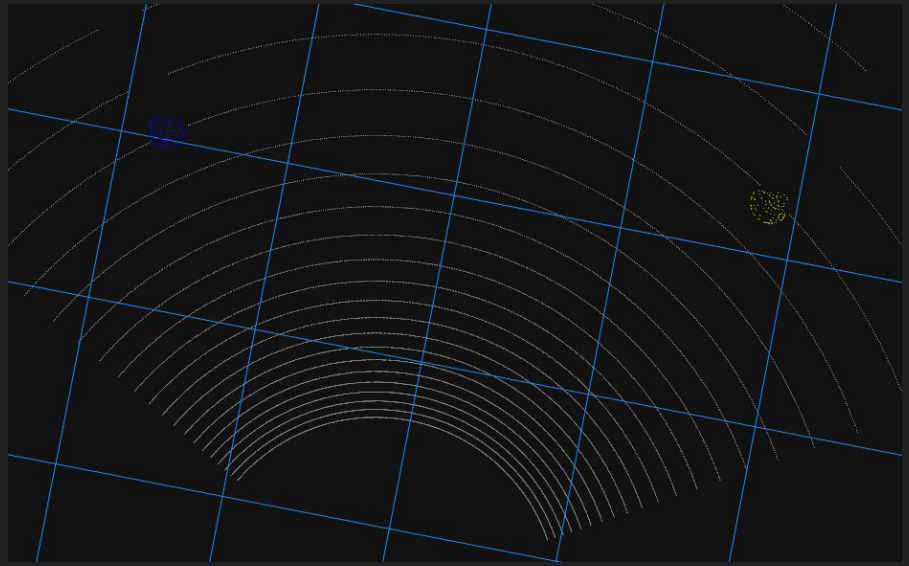




Create Occlusion Array

Input: positions of cones wrt the lidar

Output: occlusion array

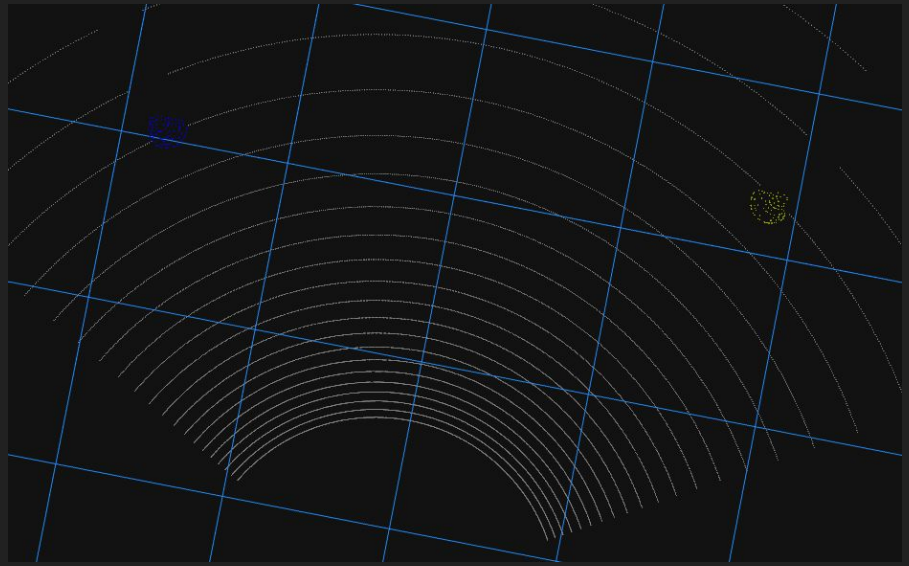


Create Occlusion Array

Input: positions of cones wrt the lidar

Output: occlusion array

$$D = \sqrt{(x^2 + y^2 + z^2)}$$



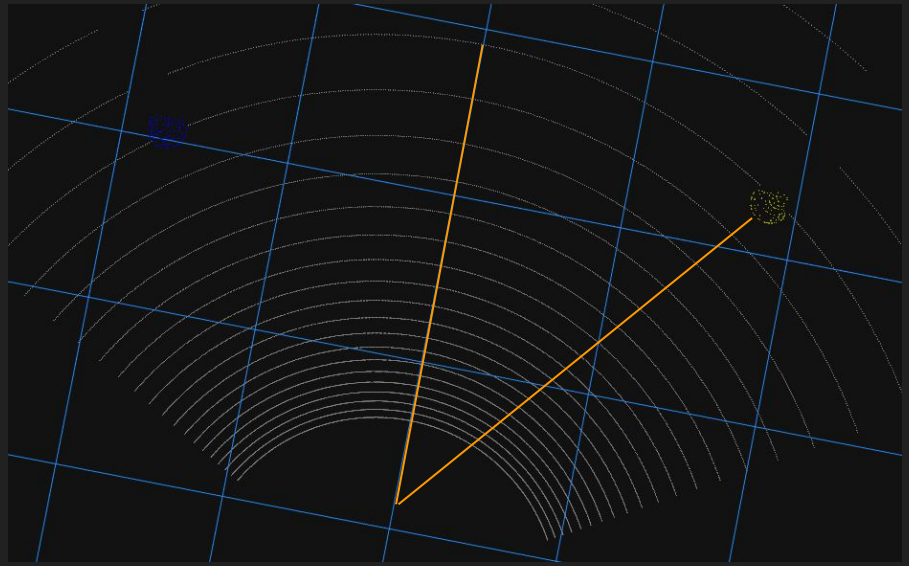
Create Occlusion Array

Input: positions of cones wrt the lidar

Output: occlusion array

$$D = \sqrt{(x^2 + y^2 + z^2)}$$

$$\theta = \text{atan2}(y, x)$$



Create Occlusion Array

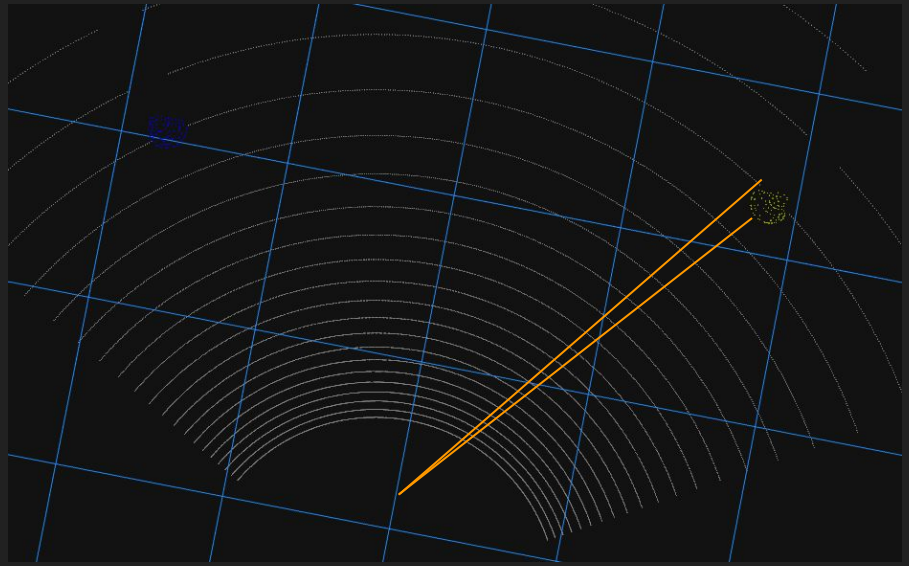
Input: positions of cones wrt the lidar

Output: occlusion array

$$D = \sqrt{(x^2 + y^2 + z^2)}$$

$$\theta = \text{atan2}(y, x)$$

$$\alpha = \arcsin(R/D)$$



Create Occlusion Array

Input: positions of cones wrt the lidar

Output: occlusion array

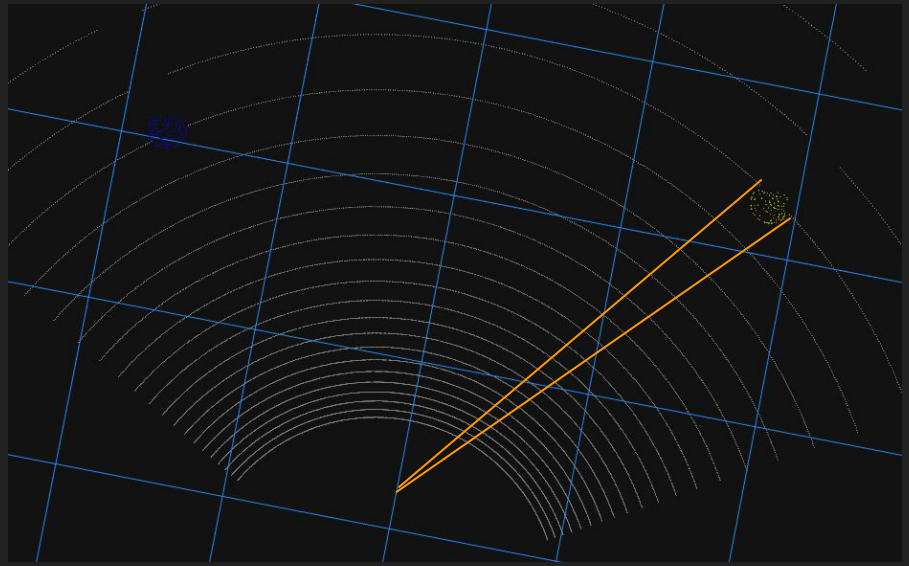
$$D = \sqrt{(x^2 + y^2 + z^2)}$$

$$\theta = \text{atan2}(y, x)$$

$$\alpha = \arcsin(R/D)$$

$$\theta\_start = \theta - \alpha$$

$$\theta\_end = \theta + \alpha$$



Create Occlusion Array

Input: positions of cones wrt the lidar

Output: occlusion array

$D = \sqrt{(x^2 + y^2 + z^2)}$

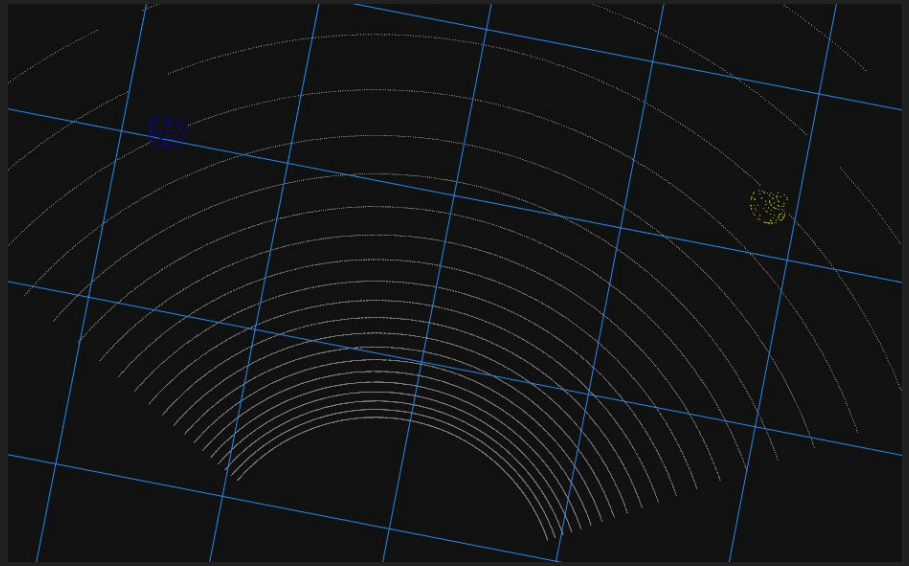
$\theta = \text{atan2}(y, x)$

$\alpha = \arcsin(R/D)$

$\theta\_start = \theta - \alpha$

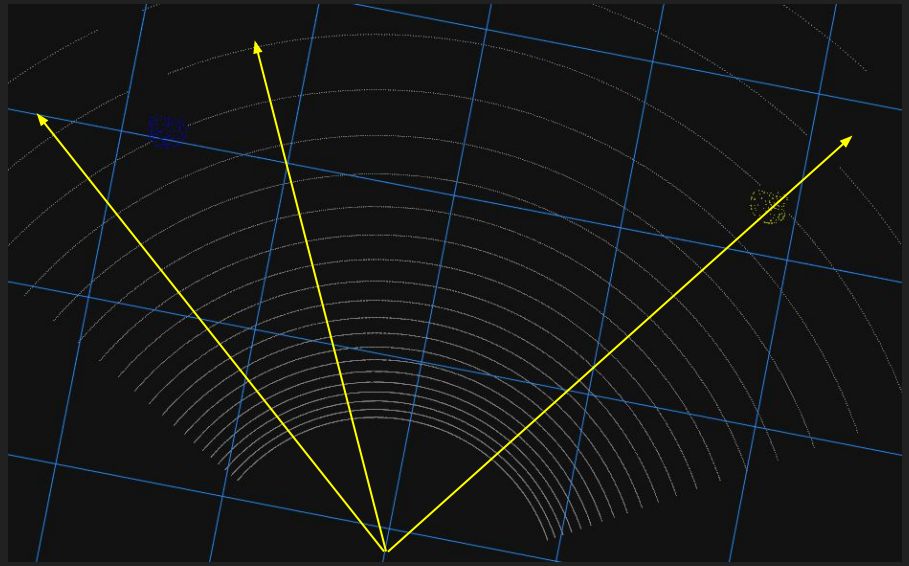
$\theta\_end = \theta + \alpha$

$\text{occlusions}[\theta\_start:\theta\_end] = \min(\text{occlusions}[\theta\_start:\theta\_end], D - R)$



Generate floor points

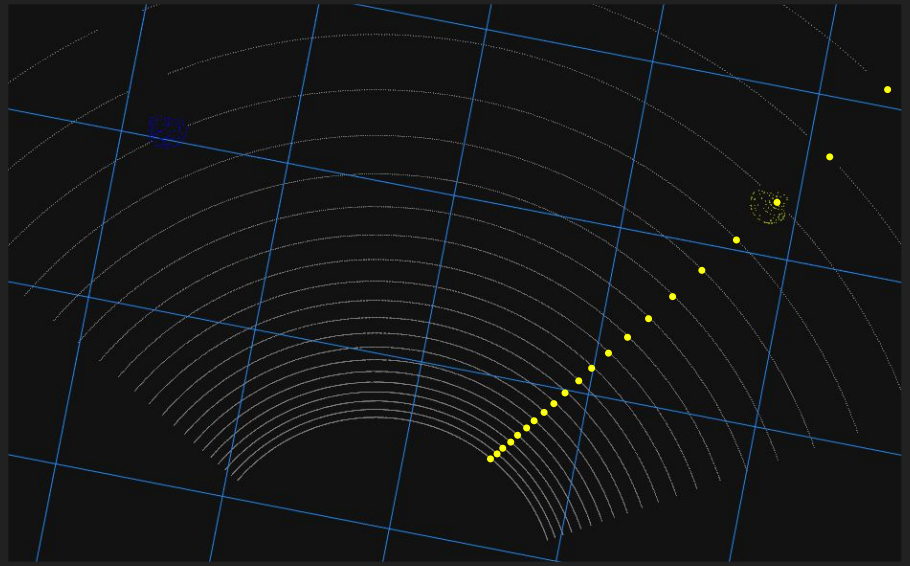
For each direction θ_h:



Generate floor points

For each direction θ_h:

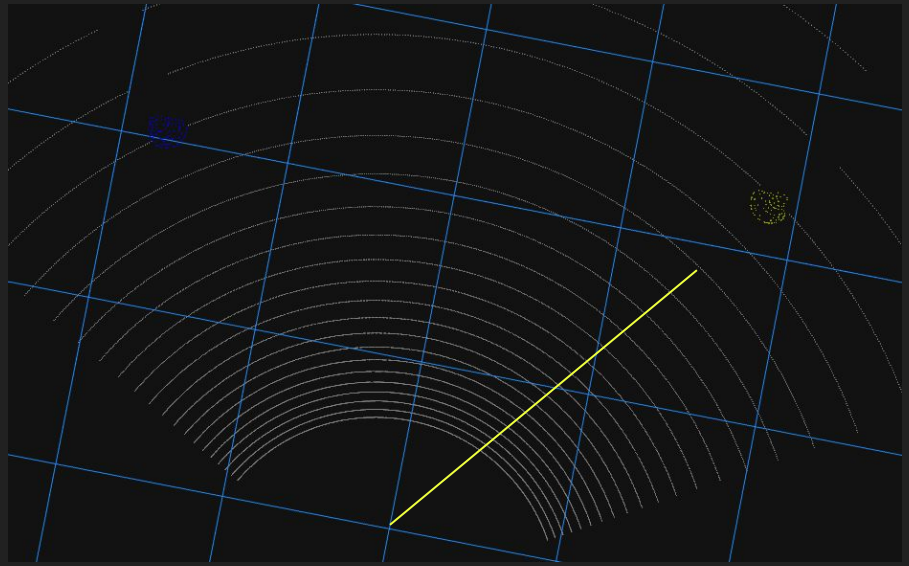
- For each channel:



Generate floor points

For each direction θ_h:

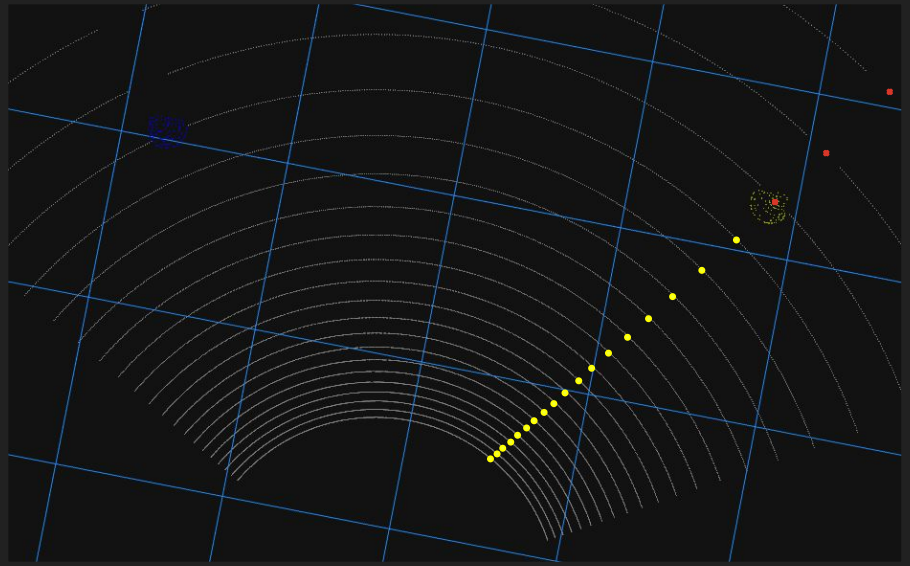
- For each channel:
  - r = H / (-tan(φ_v))



Generate floor points

For each direction θ_h:

- For each channel:
  - r = H / (-tan(φ_v))
  - If r ≤ occlusion[θ_h]:
    - x = r × cos(θ_h)
    - y = r × sin(θ_h)
    - z = 0
    - Add point (x,y,z)



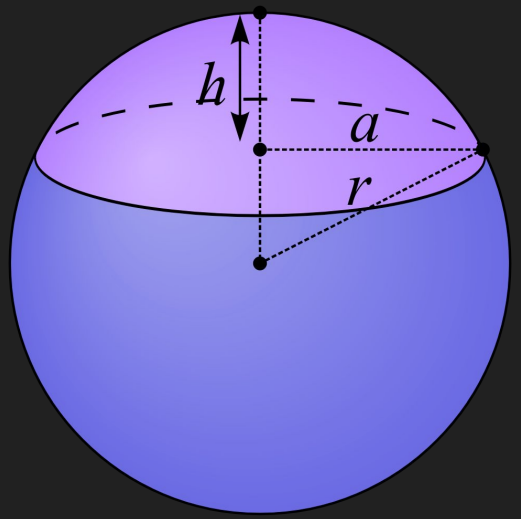
How many points per cone?

$A_{cap} = 2\pi D^2(1 - \cos(\theta))$

$A_{cone} = X_{dim} \times Z_{dim} / 2 = 0.037m^2$

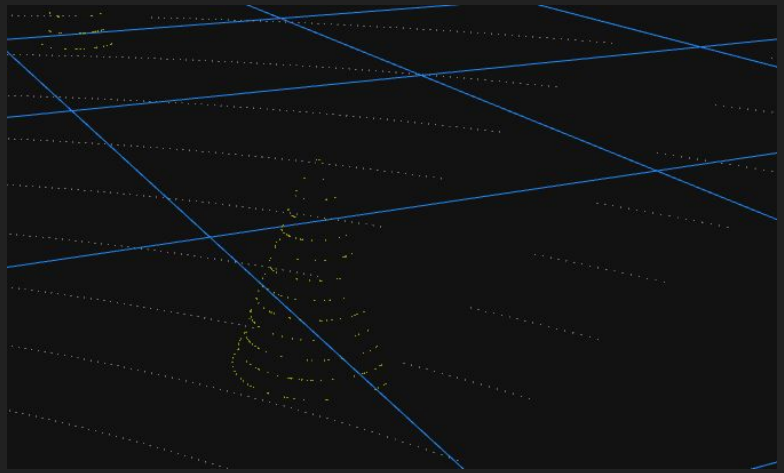
$N = total_{ray} \times A_{cone} / (2\pi D^2 (1 - \cos(\theta)))$

*some approximations has been made



Sampling on cone

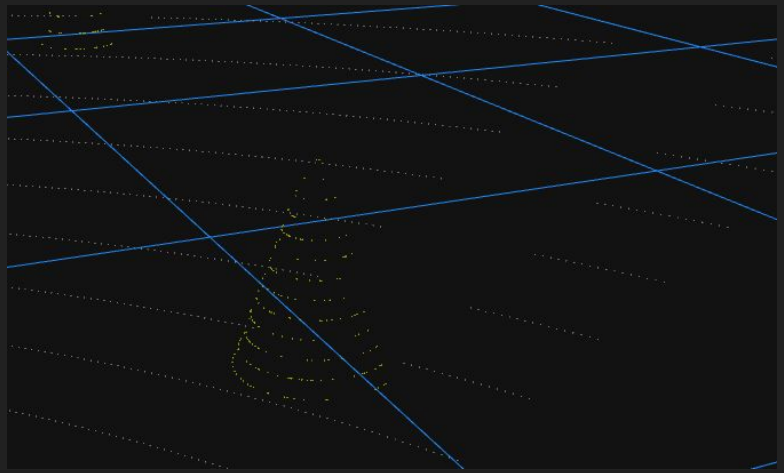
For each cone:



Sampling on cone

For each cone:

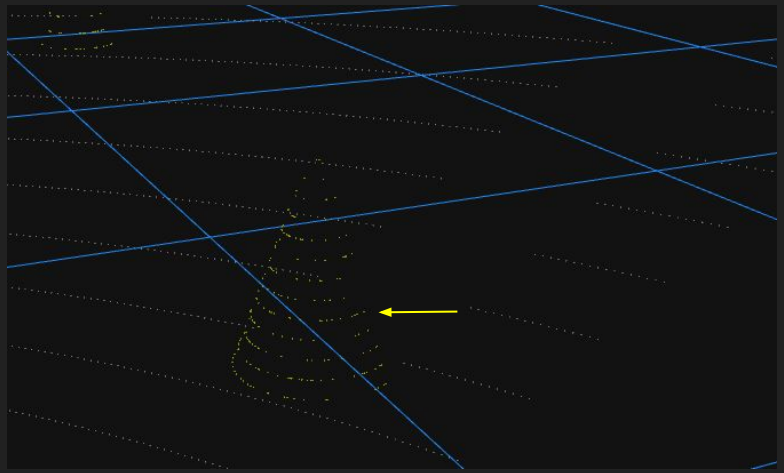
- k = number of channel obstructed



Sampling on cone

For each cone:

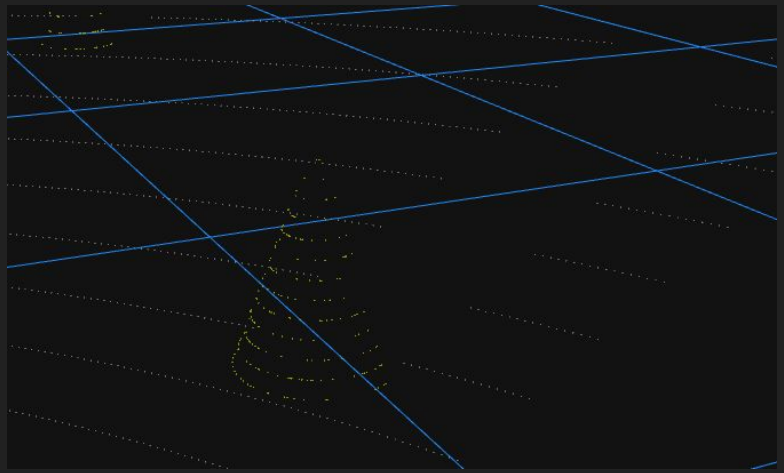
- k = number of channel obstructed
- for each point:
  - z_level ~ Weighted(k, k-1, ..., 1)
  - z = z_level × (H/k)



Sampling on cone

For each cone:

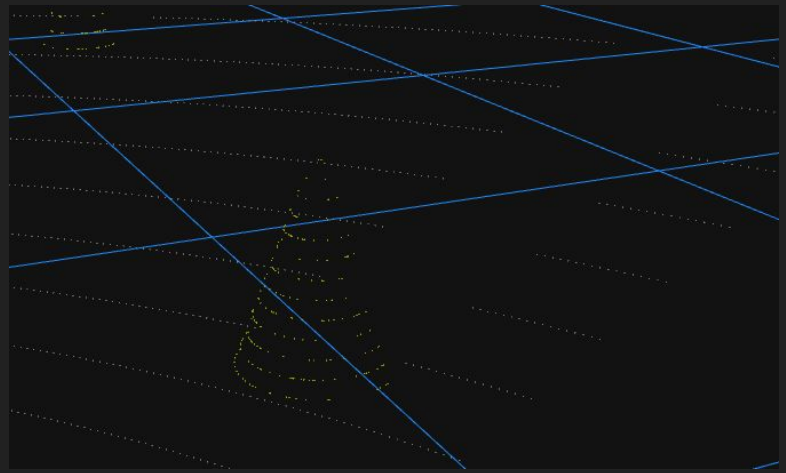
- k = number of channel obstructed
- for each point:
  - z_level ~ Weighted(k, k-1, ..., 1)
  - z = z_level × (H/k)
  - φ ~ Uniform[π+θ-α, π+θ+α] *



Sampling on cone

For each cone:

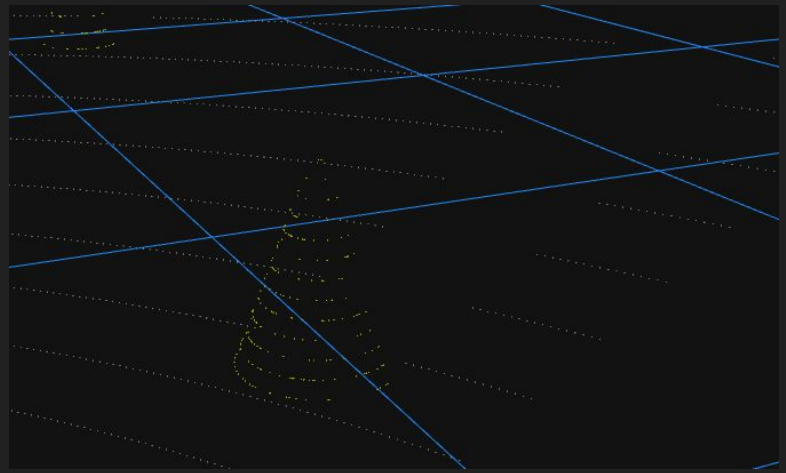
- k = number of channel obstructed
- for each point:
  - z_level ~ Weighted(k, k-1, ..., 1)
  - z = z_level × (H/k)
  - φ ~ Uniform[π+θ-α, π+θ+α]
  - x = x_cone + radius_at_z × cos(φ)
  - y = y_cone + radius_at_z × sin(φ)
  - add point (x,y,z)



Sampling on cone

For each cone:

- k = number of channel obstructed
- for each point:
  - z_level ~ Weighted(k, k-1, ..., 1)
  - z = z_level × (H/k)
  - φ ~ Uniform[π+θ-α, π+θ+α]
  - x = x_cone + radius_at_z × cos(φ)
  - y = y_cone + radius_at_z × sin(φ)
  - add point (x,y,z)

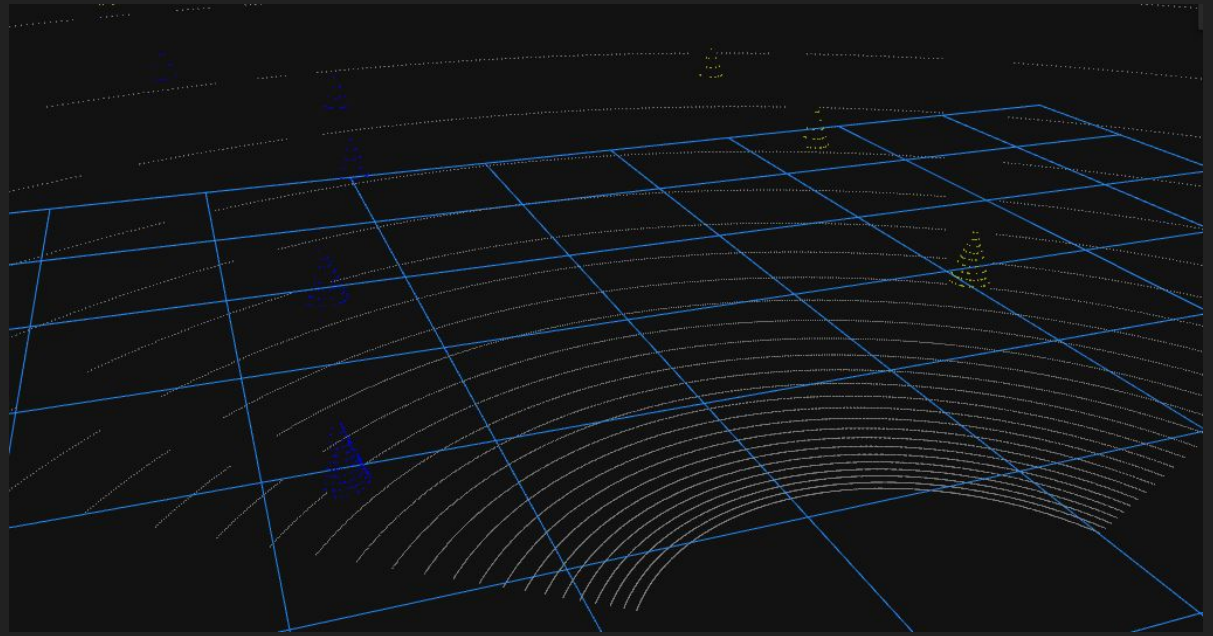


Limitations and possible improvements

Noise missing

Shadows do not close behind the cone

Partial overlapping of cones not possible

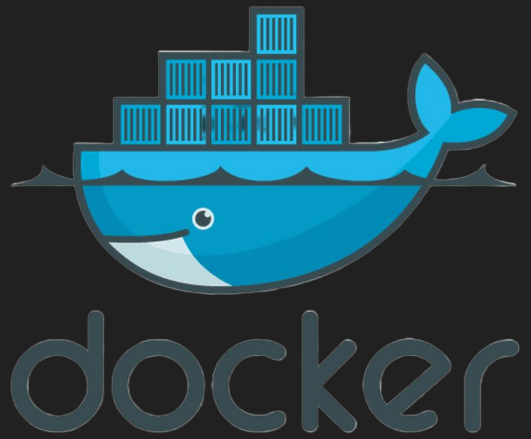


CUDA Raytracing?

Extra

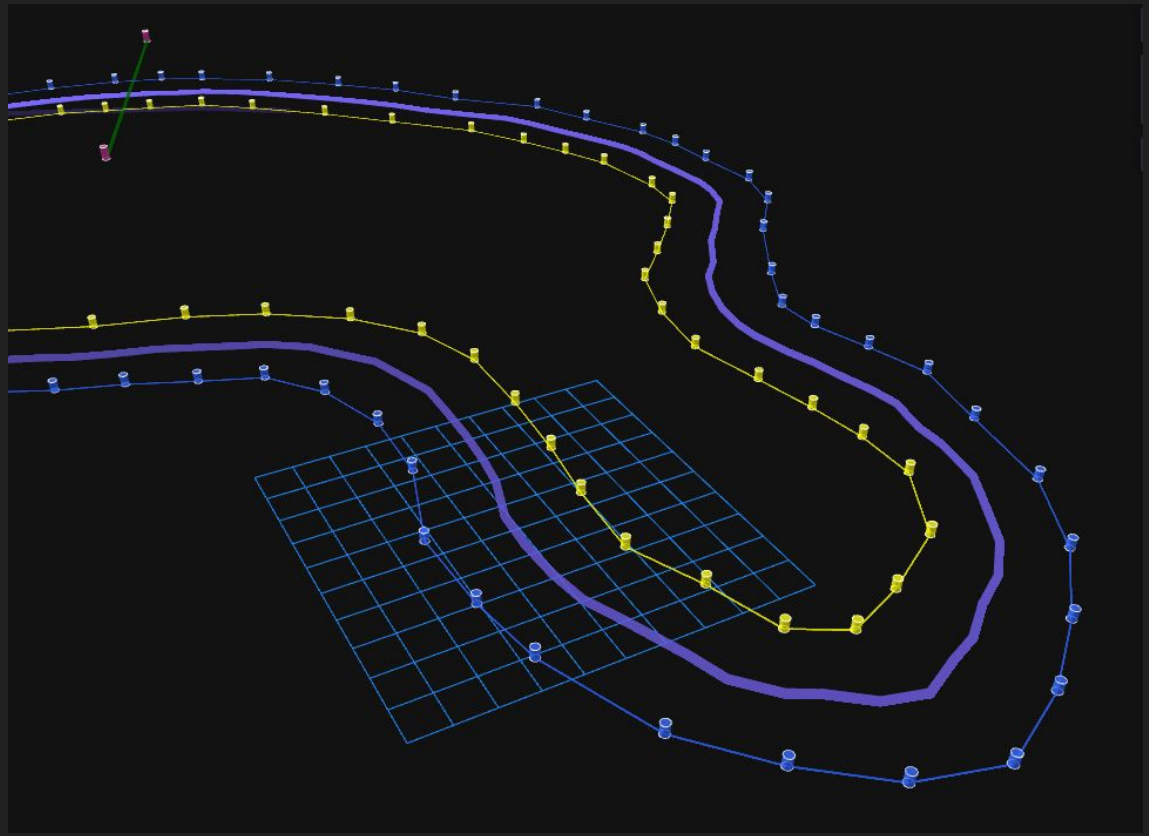
Simulations runs inside a docker container

- Avoid problem with dependencies
- Everyone in the team get an easy way of running simulations



Next Step?

track middleline?



Thanks for the attention

E-TEAM
SQUADRA CORSE
UNIVERSITÀ DI PISA